AI Powered HR People Modelling

An AI-assisted succession planning platform for TNB GENCO. It ingests HR source documents (job descriptions, talent cards, position profiles, the people model), shortlists candidates using deterministic TNB grade-band and discipline rules, then uses a Databricks-hosted LLM to predict succession readiness and explain its reasoning.

This repository holds the production React + FastAPI architecture, migrated from an earlier Streamlit prototype (retained at `app.py` for reference — see `FEATURE_MAPPING.md` and `MIGRATION_NOTES.md`).

Table of Contents

- Key Capabilities
- Architecture
- Prerequisites
- Installation
- Configuration
- Running
- Access Control
- Modules
- API Reference
- Project Structure
- Development Guide
- Troubleshooting
- Changelog
- License

Key Capabilities

- **Succession recommendation** — pick a position and receive a ranked candidate shortlist with AI-predicted readiness (`Ready Now`, `Ready in 1–2 Years`, `Ready in More Than 2 Years`) and written justification.
- **Person-to-position matching** — the inverse: pick an employee and receive the top-K positions they qualify for.
- **Rules-first shortlisting** — candidates are filtered by grade-band adjacency and education discipline *before* the model sees them, so recommendations stay inside TNB's promotion structure.
- **Document ingestion** — chunked upload of PDF and Excel sources with background parsing; 11 named sections extracted from standard talent-card PDFs.
- **Analytics dashboard** — positions by project, positions by grade, employees by grade.
- **Audit trail** — every login, upload, deletion and recommendation run recorded with actor, module, entity and outcome.
- **Export** — results downloadable as Excel or PDF from the browser.

Architecture

```
Browser (React SPA)
     |  relative /api/* calls
```

```
     v
FastAPI (backend/main.py)
     |-- /api/auth             auth
     |-- /api/uploads          ingestion + storage reconciliation
     |-- /api/analytics        dashboard aggregates
     |-- /api/recommendations  matching engine + LLM
     |-- /api/audit-logs       audit trail
     |-- /{full_path}          serves frontend/dist (SPA fallback)
     v
Storage                      Inference
 SQLite (default)             Databricks serving endpoint
 PostgreSQL (+ pgvector)      databricks-llama-4-maverick
 UC Volume / local uploads    via WorkspaceClient identity
```

The backend serves both the API and the built SPA from one origin, so no CORS configuration or proxy is needed in production.

Tech stack

Layer	Technology
Frontend	React 19, Vite 7, TypeScript 5.9, Tailwind CSS 3.4, react-router-dom 7, axios, lucide-react
Backend	Python, FastAPI, Uvicorn, Pydantic, pandas, SQLAlchemy, PyPDF2, openpyxl, tenacity
Database	SQLite (default) or PostgreSQL via pg8000, with optional pgvector
Inference	Databricks Model Serving (`databricks-llama-4-maverick`) via `databricks-sdk`
Deployment	Databricks Apps (`app.yaml`), Unity Catalog Volumes for file storage

---

Prerequisites

- **Python 3.10+**
- **Node.js 18+** and npm
- **For AI features:** a Databricks workspace context with permission to query the serving endpoint. The app uses `WorkspaceClient()` and inherits workspace identity — it does not take an API key. Outside such a context, ingestion and analytics work but recommendation runs will fail.
- **Optional:** a PostgreSQL instance if you do not want the default SQLite file.

Installation

```
git clone https://github.com/EASA-26/ai-peoplemodelling2.0-react.git
cd ai-peoplemodelling2.0-react
```

Backend:

```
pip install -r backend/requirements.txt
```

Frontend:

```
cd frontend
npm install
```

Configuration

All configuration is by environment variable. None is required for a local SQLite run.

Variable	Purpose
`ENDPOINT_NAME`	Databricks serving endpoint. Default `databricks-llama-4-maverick`
`DATABASE_URL`	Full PostgreSQL URL. Takes precedence over the `POSTGRES_*` variables
`POSTGRES_HOST`	PostgreSQL host, used when `DATABASE_URL` is unset
`POSTGRES_PORT`	PostgreSQL port
`POSTGRES_DB`	PostgreSQL database name
`POSTGRES_USER`	PostgreSQL user
`POSTGRES_PASSWORD`	PostgreSQL password
`HR_PEOPLE_APP_STORAGE_ROOT`	Explicit writable directory for the SQLite file and uploads
`HR_PEOPLE_APP_UC_VOLUME`	Unity Catalog Volume root, e.g. `/Volumes/<catalog>/<schema>/<volume>`
`VOLUME`	Fallback name for the UC Volume root

Storage resolution

The application picks the first writable location from: `HR_PEOPLE_APP_STORAGE_ROOT` → `<UC volume>/.app_state` → `<cwd>/.app_state` → `<module root>/.app_state` → `/tmp/hr_people_app/.app_state`. This chain means startup does not fail when `/Volumes` is unmounted or read-only.

When a UC Volume is configured, uploaded files are filed under these subdirectories:

Category	UC Volume subdirectory
`job-descriptions`	`JD`
`position-profiles`	`Position Profile`
`talent-cards`	`Talent Cards`
`people-model`	`Talent Profile`

Database

On startup the app creates, if missing: `job_descriptions`, `candidates`, `position_profiles`, `talent_cards` and `audit_logs`, with indexes on JD filepath and title and on audit timestamp and module. Under PostgreSQL it also attempts to enable the `vector` extension and create `vector_collections`, `source_documents`, `document_chunks` and `document_embeddings`, falling back to a non-vector schema if the extension is unavailable. These vector tables are provisioned for future semantic retrieval; no current feature depends on them. Reference SQL is in `database/postgresql/`.

Running

Production-equivalent (one origin, recommended)

```
cd frontend && npm run build && cd ..
uvicorn backend.main:app --host 0.0.0.0 --port 8000
```

Open `http://localhost:8000`. The backend serves the built SPA and the API together, so the relative `/api/*` calls resolve correctly.

Two-process development

```
uvicorn backend.main:app --reload --port 8000
```

```
cd frontend && npm run dev
```

The Vite dev server runs on `http://localhost:5173`. **Note:** the frontend calls relative `/api/*` paths and `vite.config.ts` defines no proxy, so API calls from the dev server will 404 until you add one — see Troubleshooting.

Databricks Apps

`app.yaml` installs backend dependencies, builds the frontend (falling back to an existing `dist/index.html`), and starts Uvicorn on `$DATABRICKS_APP_PORT`, defaulting to 8000.

Health checks

```
curl http://localhost:8000/api/health
curl http://localhost:8000/api/health/storage
```

The second returns the resolved database path, upload root and whether the frontend build was found — the fastest way to diagnose a storage or build problem.

Access Control

The current build has a **single administrative role with a hardcoded credential pair** (`admin` / `genco2025`, defined in `backend/app/api/routers/auth.py`). Login returns a static placeholder token, and **no API endpoint validates it** — any client that can reach the backend can call any endpoint directly.

Capability	Current state
Authentication	Single hardcoded credential pair; static placeholder token
Endpoint authorisation	None — endpoints are unauthenticated
Role separation	None — every user has full rights including bulk deletion
CORS	Open (`allow_origins=["*"]`)
Audit	Complete — all actions logged regardless of the above

> **This is suitable for demonstration only.** Before handling live talent data, integrate Databricks Apps identity or TNB SSO, enforce it per endpoint, restrict CORS to known origins, and separate HR reviewer, project owner and administrator roles.

Modules

Succession Recommendation

Given a project, position title, grade and selected job description, the engine:

1. Scores every stored JD against the position by title overlap, grade tokens, project tokens and specialty aliases, and records the best auto-detected match alongside the user's explicit selection.
2. Extracts the education and competency requirements from the JD text.
3. Filters the talent pool by **grade-band adjacency** — a map defining which candidate grades legitimately feed the target grade (`M16`, `M15`, `E16&E17`, `E14&E15`, `E12&E13` and compact variants), handling `CM`/`GM`/`M`/`E` tokens.
4. Filters by **education discipline**, resolving free-text qualifications into engineering families (civil, electrical, mechanical, electronic/C&I, general) plus law and finance, and distinguishing narrow JD requirements ("civil engineering") from broad ones ("engineering").
5. Computes a transparent **succession score** from KPI history, critical-role flag and years of experience.
6. Sends the shortlist to the LLM **in batches of 3**, parsing the returned markdown table into structured results.

`POST /api/recommendations/succession-preview` runs steps 1–5 only, returning the deterministic shortlist without an LLM call — useful for verifying the rules independently of the model.

Person-to-Position

Takes an employee name and a `top_k`, applies the same grade and discipline rules against every stored job description, and returns the best-matching positions.

Data Management

Four upload categories, each with list, single delete and bulk delete. Large files upload in chunks via `POST /api/uploads/chunk/{category}` then `POST /api/uploads/finalize/{category}`. Job descriptions and talent cards are parsed in a FastAPI background task, so the upload returns `202` immediately and content appears once processing completes. PDF text extraction is capped at 20 pages and 40,000 characters per document. Every list call reconciles database rows against files actually present in storage, so manual file removal does not leave orphan records.

Analytics and Audit

`GET /api/analytics/summary` returns counts by project, position grade and employee grade, tolerating several key spellings (`Projek`/`Project`, `Grade`/`Gred`). `GET /api/audit-logs` returns the most recent entries, default limit 200.

API Reference

All routes are prefixed `/api`. No endpoint currently requires authentication.

Auth

Endpoint	Purpose
`POST /api/auth/login`	Validate credentials; returns `logged_in`, `user` and a placeholder token

Recommendations

Endpoint	Purpose
`POST /api/recommendations/succession-preview`	Deterministic shortlist only — no LLM call
`POST /api/recommendations/succession`	Full run: shortlist plus AI readiness and reasoning
`POST /api/recommendations/person-to-position`	Top-K matching positions for one employee

Uploads

Endpoint	Purpose
`POST /api/uploads/chunk/{category}`	Upload one chunk of a large file
`POST /api/uploads/finalize/{category}`	Assemble chunks and begin processing (202)
`POST /api/uploads/job-descriptions`	Upload JD PDFs; parsed in background (202)
`GET /api/uploads/job-descriptions`	List JDs, reconciled against storage
`GET /api/uploads/job-descriptions/{jd_id}/content`	Full extracted text of one JD
`DELETE /api/uploads/job-descriptions/{jd_id}`	Delete one JD and its file if unreferenced
`DELETE /api/uploads/job-descriptions`	Delete all JDs
`POST /api/uploads/talent-cards`	Upload talent-card PDFs; parsed in background
`GET /api/uploads/talent-cards`	List talent cards
`DELETE /api/uploads/talent-cards/{card_id}`	Delete one talent card
`DELETE /api/uploads/talent-cards`	Delete all talent cards
`POST /api/uploads/position-profiles`	Upload the position profile spreadsheet
`GET /api/uploads/position-profiles`	List position profiles
`DELETE /api/uploads/position-profiles/{profile_id}`	Delete one position profile
`DELETE /api/uploads/position-profiles`	Delete all position profiles
`POST /api/uploads/people-model`	Upload the people model spreadsheet
`GET /api/uploads/people-model`	List people-model records
`DELETE /api/uploads/people-model/{candidate_id}`	Delete one candidate record
`DELETE /api/uploads/people-model`	Delete all candidate records

Analytics, Audit and Health

Endpoint	Purpose
`GET /api/analytics/summary`	Counts by project, position grade and employee grade
`GET /api/audit-logs`	Recent audit entries, default limit 200
`GET /api/health`	Liveness check
`GET /api/health/storage`	Resolved DB path, upload root, frontend build presence

Interactive OpenAPI docs are available at `/docs` when the backend is running.

Project Structure

```
.
+-- app.yaml                      Databricks Apps launch spec
+-- app.py                        Legacy Streamlit prototype (reference only)
+-- llm_client.py                 Legacy LLM helper (reference only)
+-- md2pdf.py                     Markdown to PDF renderer for the docs
+-- requirements.txt              Legacy prototype dependencies
+-- backend
|   +-- main.py                   FastAPI app, router mounts, SPA fallback
|   +-- requirements.txt          Backend dependencies
|   +-- app
|       +-- api/routers
|       |   +-- analytics.py      Dashboard aggregates
```

```
|       |   +-- audit.py           Audit log reads
|       |   +-- auth.py            Login
|       |   +-- recommendations.py Matching engine + LLM orchestration
|       |   +-- uploads.py         Ingestion, storage reconciliation, CRUD
|       +-- data/db.py             Storage resolution, schema, audit helpers
|       +-- services
|           +-- llm.py             Databricks serving client with fallbacks
|           +-- parser.py          PDF/field extraction, filename normalisation
|           +-- scoring.py         Grade bands, education buckets, scoring
+-- database/postgresql            Reference SQL schemas
+-- docs                           Value framework (source + PDF)
+-- frontend
   +-- package.json
   +-- vite.config.ts
   +-- src
       +-- App.tsx                Routes
       +-- components/layout      MainLayout shell
       +-- lib/exportUtils.ts     Excel and PDF export
       +-- pages                  Login, Dashboard, Succession,
                                  PersonToPosition, DataManagement
```

Frontend routes

Route	Page
`/login`	`LoginPage`
`/analytics`	`DashboardPage` (default redirect from `/`)
`/succession`	`SuccessionPage`
`/person-to-position`	`PersonToPositionPage`
`/data-management`	`DataManagementPage`

Development Guide

Adding an API route. Create or extend a router under `backend/app/api/routers/`, then mount it in `backend/main.py` with an `/api/...` prefix. Call `log_audit(...)` from `app.data.db` for any state-changing action.

Changing the matching rules. All domain logic lives in `backend/app/services/scoring.py` — `education_to_buckets`, `jd_required_disciplines`, `matches_requirement`, `grade_matches` and `compute_succession_score`. These are pure functions over dictionaries and are the right place to start when the rules change. Use `/succession-preview` to check a rule change without spending LLM calls.

Changing document parsing. `backend/app/services/parser.py` holds the talent-card section labels, JD education and competency regexes, and project-code normalisation. New talent-card layouts usually mean editing `ordered_labels`.

Changing the model or prompt. `backend/app/services/llm.py` wraps the Databricks endpoint. The succession prompt and its markdown-table response parsing are in `run_succession` in `recommendations.py`; the model is asked for a table with fixed headers, so changing the prompt means updating the parser alongside it.

Frontend. `npm run dev` for HMR, `npm run lint` for ESLint, `npm run build` for a type-checked production build. The `@` alias resolves to `frontend/src`.

Regenerating the documentation PDFs.

```
python md2pdf.py docs/PEOPLE_MODELLING_VALUE_FRAMEWORK.md docs/PEOPLE_MODELLING_VALUE_FRAMEWORK.pdf "People Modelling.AI - Value
```

```
python md2pdf.py README.md README.pdf "People Modelling.AI - README" "People Modelling.AI - README"
```

Keep authored tables to three columns or fewer; wider tables render with overlapping columns, and tables of five columns or more are automatically collapsed to bulleted lists.

Troubleshooting

API calls 404 in dev mode. `vite.config.ts` proxies `/api` to `http://127.0.0.1:8000`, so the backend must actually be running there. Point the proxy elsewhere with `DEV_API_TARGET`:

```
DEV_API_TARGET=http://127.0.0.1:9100 npm run dev
```

Assets 404 when served under a sub-path. The build must be told its base path, and on Windows **Git Bash mangles it**: `APP_BASE_PATH=/peoplemodelling/` becomes `C:/Program Files/Git/peoplemodelling/` through MSYS path translation, and the built `index.html` then points at that. Build from PowerShell, or disable the translation:

```
MSYS_NO_PATHCONV=1 APP_BASE_PATH=/peoplemodelling/ npm run build
```

"Frontend build not found." `backend/main.py` serves `frontend/dist`. Run `npm run build` in `frontend/`, and confirm via `GET /api/health/storage` that `frontend_found` is true.

Recommendation run returns no AI results but a shortlist appears. The rules worked and the LLM call failed — errors are caught per batch and printed. Check that the process has Databricks workspace credentials and that `ENDPOINT_NAME` is correct and reachable. Use `/succession-preview` to confirm the shortlist independently.

"No job descriptions found" or "No candidates found" (404). Both stores must be populated before a run. Upload via Data Management and confirm the list endpoints return rows.

"Candidates must have 'Education' column" (400). The people-model spreadsheet needs an `Education` column; discipline filtering cannot run without it.

Empty shortlist. Usually the grade filter. Confirm the position grade matches a key in the adjacency map in `grade_matches`, and that candidate grades use the expected `CM`/`GM`/`M`/`E` plus two-digit format.

Uploaded files vanish between restarts. Storage resolved to an ephemeral path. Check `GET /api/health/storage` and set `HR_PEOPLE_APP_STORAGE_ROOT` or `HR_PEOPLE_APP_UC_VOLUME` to a durable location.

PostgreSQL vector tables missing. The `vector` extension was unavailable and the app fell back to the non-vector schema. This is expected and affects no current feature.

Talent-card fields come back "Not provided". The PDF layout does not match the expected section labels, or the text layer is missing (a scanned image). Check the labels in `parser.py` against the document.

Changelog

1.1.0 — Background loading for data management records; audit log moved into analytics; Databricks frontend deployment fix; PostgreSQL primary database support with vector-ready schema.

1.0.0 — Initial migration from the Streamlit prototype to React + FastAPI. All 8 original pages ported with education bucket mapping, grade logic, similarity scoring and JD disciplines preserved; persistent storage replacing in-memory Streamlit state.

License

Internal TNB GENCO project. Not licensed for external distribution.